COA Practical Exam

Quick Study Guide - Experiments 2 to 10

Computer Organization and Architecture (S.E. Sem III, Mumbai University). Every experiment has: the aim in one line, short theory, the **corrected, exam-ready code**, a line-by-line explanation, a sample output or trace, and the mistakes I found in the lab-manual code. Read it top to bottom in about an hour.

Exp	Topic	Language / tool	Key idea in one line
2	ALU operations ADD, SUB, MUL	8086 MASM	ADD / SUB work on AL; MUL BL gives AX = AL x BL
3	HEX to BCD conversion	8086 MASM	Divide by 10: quotient = tens, remainder = ones; add 30H to print
4	String operations (reverse, display)	8086 MASM + INT 21H	Push chars on stack, pop them in reverse; AH=09H prints a $-string
5	BCD addition from keyboard	8086 MASM + INT 21H	Read digit (AH=01H), subtract 30H, add, divide by 10, print two digits
6	Booth's multiplication	C	Look at Q0 and Q-1: 10 = subtract, 01 = add, then arithmetic shift right
7	Restoring division	C	Shift (A,Q) left, A = A - M, if A negative then restore and Q0 = 0 else Q0 = 1
8	Direct cache mapping	C	Line = Block number MOD number of cache lines
9	Draw shapes with BIOS INT 10H	8086 MASM	Mode 13H (320x200), AH=0CH plots one pixel at (CX, DX)
10	File operations with DOS interrupts	Turbo C + INT 21H	3CH create, 40H write, 3EH close, 3DH open, 3FH read

Plan for 1 hour

- **0-10 min:** Exp 2, 3, 5 (small programs, same pattern: MOV, DIV, ADD 30H, INT 21H).
- **10-20 min:** Exp 4 (string reverse + display) and Exp 9 (pixel loops).
- **20-40 min:** Exp 6 and 7 (algorithms: practise the trace tables with a pen).
- **40-50 min:** Exp 8 and Exp 10 (short, mostly theory).
- **50-60 min:** Read the 'Mistakes fixed' table below and skim the viva PDF.

Mistakes I found in the lab-manual code (fixed in this guide)

Examiners sometimes ask 'will this run?'. These are real bugs, so write the corrected version in the exam.

Exp	Problem in the original	Fix
4	Reverse program uses CX = 12 but 'Hi C Division' has 13 characters, so the last letter 'n' is skipped. It also has no exit call (MOV AH,4CH / INT 21H) and the display program puts RET after END MAIN.	CX = 13, add the exit call, close with MAIN ENDP then END MAIN.
5	AH=09H (print string) usually returns AL = 24H ('$'), which destroys the sum held in AL.	Save the sum in CL before printing 'Result' and restore it afterwards.
6, 7	The manuals have an empty 'Code:' section.	Tested C programs are given below (verified for many inputs).
8	Program treats the address itself as the block number (fine if block size is 1 word).	Explained, plus a bonus version that also prints tag and offset.

Exp	Problem in the original	Fix
9	Triangle is not closed: left side ends at (180,50) but the right side starts at (160,50) and ends at (260,150) while the base ends at x = 240. BH (display page) is never set to 0.	Apex moved to (160,70) so both sides are 45 degrees and meet the base ends; MOV BH,0 added.
10	Checks 'handle == 0xFFFF' for errors. DOS reports errors with the Carry Flag (CF = 1) and puts the error code in AX.	Check regs.x.cflag after every int86 call.
3	Works only up to 63H (99 decimal) because it divides by 10 once. FFh = 255 would print wrong characters.	Bonus 3-digit version given.

Experiment 2 - ALU operations (ADD, SUB, MUL)

Aim: Perform addition, subtraction and multiplication on 8086 using MASM and verify in DEBUG.

Theory in short

- **ADD dest, src**: dest = dest + src. **SUB dest, src**: dest = dest - src.
- **MUL src** is unsigned. For an 8-bit source the 8086 multiplies **AL x src** and stores the 16-bit result in **AX** (AH = high byte, AL = low byte).
- For a 16-bit source: AX x src gives a 32-bit result in **DX:AX**.
- The result of ADD/SUB also updates flags (ZF, CF, SF, OF...).

Program

```
.MODEL SMALL
.STACK 100H
.DATA
NUM1 DB 05H
NUM2 DB 03H
.CODE
MAIN PROC
    MOV AX, @DATA      ; load address of data segment
    MOV DS, AX         ; DS now points to our data

    MOV AL, NUM1       ; AL = 05H
    ADD AL, NUM2       ; AL = 05H + 03H = 08H

    MOV AL, NUM1
    SUB AL, NUM2       ; AL = 05H - 03H = 02H

    MOV AL, NUM1
    MOV BL, NUM2
    MUL BL             ; AX = AL x BL = 0FH (000FH)

    MOV AH, 4CH        ; DOS function: terminate program
    INT 21H
MAIN ENDP
END MAIN
```

Line-by-line

Line	Meaning
.MODEL SMALL	One code segment + one data segment (simplest memory model).
.STACK 100H	Reserve 256 bytes of stack.
.DATA / DB	Define byte variables NUM1 = 05H and NUM2 = 03H.
MOV AX,@DATA ; MOV DS,AX	Initialise DS. A segment register cannot be loaded directly with a constant, so go through AX.
ADD AL,NUM2	Adds memory byte NUM2 to AL. Result 08H stays in AL.
SUB AL,NUM2	Subtracts NUM2 from AL. Result 02H.
MUL BL	AX = AL x BL. 5 x 3 = 15 = 0FH, so AX = 000FH.
MOV AH,4CH ; INT 21H	Return to DOS (exit).

Verifying in DEBUG

```
C:\> DEBUG EXP2.EXE
-U            (unassemble: see the instructions)
-T            (trace: execute one instruction at a time)
-R            (show registers after every step: AL = 08, then 02, then AX = 000F)
-G            (run the whole program)
-Q            (quit)
```

Expected result: after ADD AL=08H, after SUB AL=02H, after MUL AX=000FH.

Experiment 3 - HEX to BCD conversion

Aim: Convert a hexadecimal number stored in memory to its decimal (BCD) digits and display them.

Theory in short

- **Hex** is base 16. 25H = 2x16 + 5 = **37 decimal**. FFh = 255.
- **BCD** stores each decimal digit in 4 bits. *Packed BCD*: two digits per byte (37 decimal = 37H). *Unpacked BCD*: one digit per byte (03H, 07H).
- **Method:** divide the number by 10. Quotient = tens digit, remainder = ones digit. Add 30H to get the ASCII code so INT 21H can print it.
- **DIV BL** (8-bit divisor): the dividend is AX, so AH must be cleared first. Quotient goes to **AL**, remainder to **AH**.

Program

```
.MODEL SMALL
.STACK 100H
.DATA
NUM DB 25H          ; hex number to convert (= 37 decimal)
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    MOV AL, NUM     ; AL = 25H = 37
    MOV AH, 00H     ; clear AH so that AX = 0025H
    MOV BL, 10
    DIV BL          ; AL = 37 / 10 = 3 (tens), AH = 37 MOD 10 = 7 (ones)
    MOV BH, AH      ; save ones digit (AH will be overwritten by the next MOV AH,..)

    ADD AL, 30H     ; 3 -> '3'
    MOV DL, AL
    MOV AH, 02H     ; DOS: print character in DL
    INT 21H

    ADD BH, 30H     ; 7 -> '7'
    MOV DL, BH
    MOV AH, 02H
    INT 21H

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Output: 37. This version is correct only for numbers up to 63H (99). That is enough for the practical; the bonus below handles up to FFh.

Bonus: 3-digit version (up to FFh = 255)

```
    MOV AL, NUM
    MOV AH, 00H
    MOV BL, 100
    DIV BL          ; AL = hundreds, AH = remainder (less than 100)
    MOV CL, AH      ; keep remainder safe
    ADD AL, 30H
    MOV DL, AL
    MOV AH, 02H
    INT 21H         ; print hundreds
    MOV AL, CL
    MOV AH, 00H
    MOV BL, 10
    DIV BL          ; AL = tens, AH = ones
    MOV BH, AH
    ADD AL, 30H
    MOV DL, AL
    MOV AH, 02H
    INT 21H         ; print tens
    ADD BH, 30H
    MOV DL, BH
    MOV AH, 02H
    INT 21H         ; print ones     (for NUM = FFh the screen shows 255)
```

If the examiner asks for **packed BCD** instead of printing: after DIV BL do `MOV CL,4 / SHL AL,CL / OR AL,AH`. For 25H this gives AL = 37H (tens in the high nibble, ones in the low nibble).

Experiment 4 - String operations (reverse and display)

Aim: Implement string operations in 8086 using DOS interrupts (MASM).

Theory in short

- A string is a sequence of bytes or words in consecutive memory. DOS prints a string with **INT 21H, AH=09H**, and the string must end with **$**.
- **SI** = source index (reads DS:SI), **DI** = destination index (writes ES:DI). The **direction flag** (CLD / STD) decides whether SI/DI increase or decrease.
- **MOVSB** copies DS:SI to ES:DI. **LODSB** loads DS:SI into AL. **STOSB** stores AL at ES:DI. **SCASB** compares AL with ES:DI. **CMPSB** compares DS:SI with ES:DI.
- **REP** repeats the string instruction while CX is not 0 (decrements CX each time). **REPE/REPZ** repeat while CX is not 0 *and* ZF = 1. **REPNE/REPNZ** repeat while CX is not 0 *and* ZF = 0.
- The 'D' versions (MOVSD, LODSD...) are 32-bit and exist only on 80386 and later, not on the 8086. (MOVD in the lab handout is a typo for MOVSD.)

Part A - Reverse a string (stack method)

Algorithm: push every character on the stack one by one, then pop them. The stack is LIFO (last in, first out) so characters come out reversed.

```
.MODEL SMALL
.STACK 100H
.DATA
STR1 DB 'Hi C Division'      ; 13 characters
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV SI, OFFSET STR1      ; SI points to first character
    MOV CX, 13                 ; number of characters (counter)
L1: MOV AL, [SI]               ; take one character
    MOV AH, 00H                ; make it a 16-bit value (PUSH needs 16 bits)
    PUSH AX                    ; store it on the stack
    INC SI                     ; next character
    LOOP L1                    ; CX = CX - 1, repeat while CX is not 0

    MOV CX, 13
L2: POP DX                     ; DL = last character pushed
    MOV AH, 02H                ; DOS: print character in DL
    INT 21H
    LOOP L2

    MOV AH, 4CH                ; exit to DOS
    INT 21H
MAIN ENDP
END MAIN
```

Output: noisiviD C iH
Why the manual's code was wrong: it used CX = 12, so it printed 'oisiviD C iH' (the last letter 'n' was missed), and it had no exit call. Count the characters: H-i-space-C-space-D-i-v-i-s-i-o-n = 13.

Part B - Display a string

```
.MODEL SMALL
.STACK 100H
.DATA
K1 DB 'Hello This is C Division here $'    ; $ marks the end of string
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    LEA DX, K1          ; DX = address of the string
    MOV AH, 09H         ; DOS: display string ending with $
    INT 21H
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Output: Hello This is C Division here

Bonus: copy a string with string instructions (good for viva)

```
.MODEL SMALL
.STACK 100H
.DATA
SRC1 DB 'HELLO$'
DST1 DB 6 DUP(?)
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX
    MOV ES, AX          ; MOVSB writes to ES:DI, so ES must be set too
    LEA SI, SRC1
    LEA DI, DST1
    MOV CX, 6           ; bytes to copy (includes $)
    CLD                 ; direction flag = 0: SI and DI increase
    REP MOVSB           ; copy 6 bytes
    LEA DX, DST1
    MOV AH, 09H
    INT 21H             ; prints HELLO
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Experiment 5 - BCD addition using keyboard input (INT 21H)

Aim: Add two BCD digits entered from the keyboard and show the result on the monitor.

Theory in short

Interrupt	AH	Purpose
INT 21H	01H	Read one character from keyboard (with echo). Character comes back in AL.
INT 21H	02H	Display the character in DL.
INT 21H	09H	Display the $-terminated string at DS:DX.
INT 21H	4CH	Terminate the program.

- Keyboard gives ASCII. The character '5' is 35H, so **subtract 30H** to get the number 5.
- Two digits added can reach 18 (9+9), which needs two output digits. Divide the sum by 10: quotient = tens, remainder = ones. Add 30H to each and print.
- The lab version divides by 10 instead of using DAA (decimal adjust after addition).

Program (corrected)

```
.MODEL SMALL
.STACK 100H
.DATA
MSG1 DB 'Enter First BCD Number (0-9): $'
MSG2 DB 13,10,'Enter Second BCD Number (0-9): $'     ; 13,10 = carriage return + line feed
MSG3 DB 13,10,'Result = $'
.CODE
MAIN PROC
    MOV AX, @DATA
    MOV DS, AX

    LEA DX, MSG1
    MOV AH, 09H
    INT 21H                 ; show prompt 1
    MOV AH, 01H
    INT 21H                 ; AL = first key pressed (ASCII)
    SUB AL, 30H             ; ASCII -> number
    MOV BL, AL              ; keep first number in BL

    LEA DX, MSG2
    MOV AH, 09H
    INT 21H                 ; show prompt 2
    MOV AH, 01H
    INT 21H                 ; AL = second key (ASCII)
    SUB AL, 30H
    ADD AL, BL              ; AL = sum (0 to 18)
    MOV CL, AL              ; <<< FIX: save the sum, AH=09H overwrites AL with '$'

    LEA DX, MSG3
    MOV AH, 09H
    INT 21H                 ; print "Result = "

    MOV AL, CL              ; <<< FIX: restore the sum
    MOV AH, 00H             ; clear AH before DIV
    MOV BL, 10
    DIV BL                  ; AL = tens, AH = ones
    MOV BH, AH              ; save ones

    ADD AL, 30H
    MOV DL, AL
    MOV AH, 02H
    INT 21H                 ; print tens digit

    MOV DL, BH
    ADD DL, 30H
    MOV AH, 02H
    INT 21H                 ; print ones digit

    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Sample run

```
Enter First BCD Number (0-9): 7
Enter Second BCD Number (0-9): 8
Result = 15
```

Note: a sum like 7 + 2 prints as 09 (tens digit 0). The 'two 8-bit BCD numbers' in the aim means two one-digit BCD numbers entered separately; if asked about 2-digit inputs, explain you would read two digits for each number and use packed BCD with DAA.

Experiment 6 - Booth's multiplication algorithm

Aim: Multiply two signed numbers (2's complement) using Booth's algorithm.

Theory in short

- Registers: **M** = multiplicand, **Q** = multiplier, **A** = accumulator (starts 0), **Q-1** = extra bit (starts 0), **Count** = n (number of bits).
- Each step looks at the pair (**Q0, Q-1**): **10** means A = A - M; **01** means A = A + M; **00 or 11** means do nothing. Then do an **arithmetic shift right (ASR)** on A, Q, Q-1 together (the sign bit of A is kept).
- After n steps the 2n-bit product is A and Q joined together (A Q).
- Works for positive and negative numbers with no special sign handling, and does fewer additions when the multiplier has long runs of 1s or 0s.

Booth's recoding table

Q0	Q-1	Operation
0	0	Only arithmetic shift right
0	1	A = A + M, then shift
1	0	A = A - M, then shift
1	1	Only arithmetic shift right

C program (tested)

```
#include <stdio.h>

void printbits(int v, int n) {              /* prints n bits of v */
    for (int i = n - 1; i >= 0; i--) printf("%d", (v >> i) & 1);
}

int main() {
    int n = 8;                              /* number of bits */
    int M, Q, A = 0, Q1 = 0;                /* Q1 is Q-1 */
    int mask = (1 << n) - 1;                /* 0xFF for 8 bits */
    printf("Enter multiplicand (-128 to 127): ");  scanf("%d", &M);
    printf("Enter multiplier   (-128 to 127): ");  scanf("%d", &Q);
    int origM = M, origQ = Q;
    Q = Q & mask;                           /* keep Q as an 8-bit pattern */

    printf("\nStep   A          Q          Q-1  Operation\n");
    for (int c = n; c > 0; c--) {
        int q0 = Q & 1;                     /* last bit of Q */
        const char *op = "No operation";
        if (q0 == 1 && Q1 == 0)      { A = A - M; op = "A = A - M"; }   /* pair 10 */
        else if (q0 == 0 && Q1 == 1) { A = A + M; op = "A = A + M"; }   /* pair 01 */

        /* arithmetic shift right of (A, Q, Q-1) */
        Q1 = Q & 1;                                      /* old Q0 goes to Q-1 */
        Q  = ((Q >> 1) | ((A & 1) << (n - 1))) & mask;  /* A's last bit enters Q's top */
        A  = A >> 1;                                     /* A shifts, sign is kept */

        printf("%-6d ", n - c + 1);
        printbits(A & mask, n); printf(" ");
        printbits(Q, n);        printf("   %d    %s\n", Q1, op);
    }
    int product = ((A & mask) << n) | Q;    /* A and Q joined = 2n-bit product */
    if (product & (1 << (2 * n - 1))) product -= (1 << (2 * n));   /* make it signed */
    printf("\n%d x %d = %d\n", origM, origQ, product);
    return 0;
}
```

Worked example by hand (4 bits): 3 x (-2)

M = 0011 (3), Q = 1110 (-2). Product should be -6 = 11111010 in 8 bits. The columns are A, Q and Q-1 **after** the shift of that step.

Step	Q0 Q-1	Operation	A	Q	Q-1
Start			0000	1110	0
1	0 0	Shift only	0000	0111	0
2	1 0	A = A - M = 1101, then shift	1110	1011	1
3	1 1	Shift only	1111	0101	1
4	1 1	Shift only	1111	1010	1

Product = A Q = 1111 1010 = -6 (correct).

Output of the C program for 5 x -4

```
Step   A          Q          Q-1  Operation
1      00000000 01111110   0    No operation
2      00000000 00111111   0    No operation
3      11111101 10011111   1    A = A - M
4      11111110 11001111   1    No operation
5      11111111 01100111   1    No operation
6      11111111 10110011   1    No operation
7      11111111 11011001   1    No operation
8      11111111 11101100   1    No operation

5 x -4 = -20
```

Experiment 7 - Restoring division algorithm

Aim: Divide two unsigned binary numbers using the restoring method and get the quotient and remainder.

Theory in short

- Registers: **A** = partial remainder (starts 0), **Q** = dividend (becomes the quotient), **M** = divisor, **n** = number of bits.
- (A, Q) is treated as one 2n-bit register. Each step: **shift (A,Q) left**, then **A = A - M**.
- If A is **negative** (sign bit 1): set Q0 = 0 and **restore** A by adding M back. If A is **not negative**: set Q0 = 1.
- Repeat n times. At the end **Q = quotient** and **A = remainder**.
- Called 'restoring' because a wrong subtraction is undone (restored) immediately.

C program (tested for all 4-bit inputs)

```
#include <stdio.h>

void printbits(int v, int n) {
    for (int i = n - 1; i >= 0; i--) printf("%d", (v >> i) & 1);
}

int main() {
    int n = 4;                          /* number of bits */
    int A = 0, Q, M;
    printf("Enter dividend (0-15): ");  scanf("%d", &Q);
    printf("Enter divisor  (1-15): ");  scanf("%d", &M);
    if (M == 0) { printf("Divide by zero!\n"); return 1; }

    printf("\nStep  A      Q     Operation\n");
    for (int i = 1; i <= n; i++) {
        /* shift (A,Q) left by 1: top bit of Q moves into A */
        A = (A << 1) | ((Q >> (n - 1)) & 1);
        Q = (Q << 1) & ((1 << n) - 1);
        A = A - M;                      /* A = A - M */
        if (A < 0) {                    /* negative: sign bit = 1 */
            A = A + M;                  /* restore A; Q0 stays 0 */
            printf("%-5d ", i); printbits(A, n + 1); printf(" "); printbits(Q, n);
            printf("  A<0, Q0=0, restore A\n");
        } else {
            Q = Q | 1;                  /* Q0 = 1 */
            printf("%-5d ", i); printbits(A, n + 1); printf(" "); printbits(Q, n);
            printf("  A>=0, Q0=1\n");
        }
    }
    printf("\nQuotient = %d, Remainder = %d\n", Q, A);
    return 0;
}
```

Trace for 11 / 3 (dividend 1011, divisor 0011)

Step	Shift (A,Q) left	A - M	Sign	Action	A	Q
Start					00000	1011
1	A=00001, Q=011_	00001 - 00011 = negative	1	Q0 = 0, restore A	00001	0110
2	A=00010, Q=110_	00010 - 00011 = negative	1	Q0 = 0, restore A	00010	1100
3	A=00101, Q=100_	00101 - 00011 = 00010	0	Q0 = 1	00010	1001
4	A=00101, Q=001_	00101 - 00011 = 00010	0	Q0 = 1	00010	0011

Quotient = Q = 0011 = 3, Remainder = A = 00010 = 2. Check: 3 x 3 + 2 = 11.

Program output

```
Step  A      Q     Operation
1     00001 0110  A<0, Q0=0, restore A
2     00010 1100  A<0, Q0=0, restore A
3     00010 1001  A>=0, Q0=1
4     00010 0011  A>=0, Q0=1

Quotient = 3, Remainder = 2
```

Experiment 8 - Direct mapping of cache memory

Aim: Implement direct mapping and find the cache line for a given memory address.

Theory in short

- **Cache** is small, very fast memory between the CPU and main memory (RAM). Data found in cache = **hit**; not found = **miss**. Hit ratio = hits / total accesses.
- **Mapping** decides where a main-memory block may be placed in cache. Three types: **direct, associative, set-associative**.
- **Direct mapping:** each memory block goes to exactly one cache line: **line = (block number) MOD (number of cache lines)**.
- The address is split into **Tag | Line | Offset**. Tag identifies which block currently sits in the line, Line selects the cache line, Offset selects the byte/word in the block.
- Drawback: two blocks that map to the same line keep replacing each other (conflict misses), even if other lines are empty.

Program (from the manual)

```
#include <stdio.h>
int main() {
    int address, cacheLines, line;
    printf("Enter Physical Address: ");
    scanf("%d", &address);
    printf("Enter Number of Cache Lines: ");
    scanf("%d", &cacheLines);
    line = address % cacheLines;          /* direct mapping formula */
    printf("\nPhysical Address = %d", address);
    printf("\nCache Line = %d\n", line);
    return 0;
}
```

Sample: address 25, cache lines 8 gives 25 MOD 8 = 1, so Cache Line = 1. (Here 'address' is treated as the block number, i.e. block size = 1 word.)

Bonus: with block size, tag and offset (tested)

```
#include <stdio.h>
int main() {
    int address, blockSize, cacheLines;
    printf("Enter Address: ");               scanf("%d", &address);
    printf("Enter Block Size (words): ");    scanf("%d", &blockSize);
    printf("Enter Number of Cache Lines: "); scanf("%d", &cacheLines);
    int offset = address % blockSize;        /* position inside the block */
    int block  = address / blockSize;        /* block number in main memory */
    int line   = block % cacheLines;         /* cache line */
    int tag    = block / cacheLines;         /* identifies which block is in the line */
    printf("Block No = %d\nCache Line = %d\nTag = %d\nOffset = %d\n", block, line, tag, offset);
    return 0;
}
```

Example: address 100, block size 4, 8 lines: block = 25, line = 25 MOD 8 = 1, tag = 25 / 8 = 3, offset = 0.

Experiment 9 - Drawing shapes with BIOS INT 10H (graphics)

Aim: Draw a rectangle and a triangle in graphics mode using BIOS video services (MASM, run in DOSBox).

Theory in short

- **BIOS INT 10H** is the video service. **AX = 0013H** sets **mode 13H**: 320 x 200 pixels, 256 colours (X = 0 to 319, Y = 0 to 199). **AX = 0003H** goes back to text mode.
- **AH = 0CH** plots one pixel (write pixel).

Register	Purpose
AH	0CH = function number (write pixel)
AL	Pixel colour (04H = red)
BH	Display page (0)
CX	X coordinate (column)
DX	Y coordinate (row)

- A line is drawn with a loop: plot a pixel, change CX (horizontal) or DX (vertical), compare with the end value, jump back.
- INT 16H, AH = 00H waits for a key press so the picture stays on screen.

Program 1 - Rectangle (corners (80,50) and (240,150))

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AX, 0013H      ; graphics mode 13H (320x200, 256 colours)
    INT 10H
    MOV BH, 00H        ; display page 0 (added: BH was never set in the manual)

    MOV CX, 80         ; ---- top line: x = 80..240 at y = 50
    MOV DX, 50
TOP:
    MOV AH, 0CH        ; write pixel
    MOV AL, 04H        ; colour red
    INT 10H
    INC CX             ; next pixel to the right
    CMP CX, 240
    JBE TOP            ; repeat while CX is below or equal to 240

    MOV CX, 80         ; ---- bottom line: x = 80..240 at y = 150
    MOV DX, 150
BOTTOM:
    MOV AH, 0CH
    MOV AL, 04H
    INT 10H
    INC CX
    CMP CX, 240
    JBE BOTTOM

    MOV CX, 80         ; ---- left line: y = 50..150 at x = 80
    MOV DX, 50
LEFT:
    MOV AH, 0CH
    MOV AL, 04H
    INT 10H
    INC DX             ; next pixel downwards
    CMP DX, 150
    JBE LEFT

    MOV CX, 240        ; ---- right line: y = 50..150 at x = 240
    MOV DX, 50
RIGHT:
    MOV AH, 0CH
    MOV AL, 04H
    INT 10H
    INC DX
    CMP DX, 150
    JBE RIGHT

    MOV AH, 00H        ; wait for a key
    INT 16H
    MOV AX, 0003H      ; back to text mode
    INT 10H
    MOV AH, 4CH        ; exit
    INT 21H
MAIN ENDP
END MAIN
```

Program 2 - Triangle (corrected: base (80,150)-(240,150), apex (160,70))

```
.MODEL SMALL
.STACK 100H
.CODE
MAIN PROC
    MOV AX, 0013H
    INT 10H
    MOV BH, 00H

    MOV CX, 80          ; ---- base: x = 80..240 at y = 150
    MOV DX, 150
BASE:
    MOV AH, 0CH
    MOV AL, 04H
    INT 10H
    INC CX
    CMP CX, 240
    JBE BASE

    MOV CX, 80          ; ---- left side: go right and up (x+1, y-1) to the apex
    MOV DX, 150
LEFT:
    MOV AH, 0CH
    MOV AL, 04H
    INT 10H
    INC CX
    DEC DX
    CMP DX, 70          ; <<< FIX: was 50, which overshot the apex
    JAE LEFT            ; (unsigned compare: repeat while DX is 70 or more)

    MOV CX, 160         ; ---- right side: start at apex (160,70), go right and down (x+1, y+1)
    MOV DX, 70          ; <<< FIX: was 50
RIGHT:
    MOV AH, 0CH
    MOV AL, 04H
    INT 10H
    INC CX
    INC DX
    CMP DX, 150
    JBE RIGHT

    MOV AH, 00H
    INT 16H
    MOV AX, 0003H
    INT 10H
    MOV AH, 4CH
    INT 21H
MAIN ENDP
END MAIN
```

Why the fix: with sides moving 1 pixel across per 1 pixel up, the apex must be half the base width (80 pixels) above the base. The base is 160 wide (80 to 240) so the apex is at x = 160 and y = 150 - 80 = 70. In the manual the left side reached (180,50) while the right side started at (160,50) and ended at (260,150), so the triangle did not close.

Output: a red rectangle (160 x 100 pixels) and a red triangle outline in the middle of the screen; any key returns to DOS.

Experiment 10 - File operations using DOS interrupts in C

Aim: Create, write, close, open, read and close a file using INT 21H calls inside a C program (Turbo C / DOSBox).

Theory in short

- File services are in **INT 21H**; the function number goes in **AH**.
- A file is identified by a **file handle** (a small number returned by create/open) that you pass to read, write and close.
- In C, **int86(0x21, &in, &out)** from **dos.h** loads the registers from **union REGS** (fields regs.h.ah, regs.x.bx ...), runs the interrupt and copies the result registers back.
- **Error handling:** on error DOS sets the **Carry Flag (CF = 1)** and puts an error code in AX. In C this flag is **regs.x.cflag**.

Function (AH)	Action	Input registers	Returns
3CH	Create file	CX = attribute (0 = normal), DS:DX = filename (ends with 0)	AX = handle
3DH	Open file	AL = mode (0 read, 1 write, 2 read/write), DS:DX = filename	AX = handle
40H	Write to file	BX = handle, CX = bytes to write, DS:DX = buffer	AX = bytes written
3FH	Read from file	BX = handle, CX = bytes to read, DS:DX = buffer	AX = bytes read (0 = end of file)
3EH	Close file	BX = handle	CF set on error

Program (corrected: errors checked with the carry flag)

```
#include <dos.h>
#include <stdio.h>
#include <string.h>

void main() {
    unsigned int handle;
    char buffer[100];
    char filename[] = "test.txt";
    char data[] = "Hello DOS file operations!\r\n";    /* 28 bytes */
    union REGS regs;

    /* 1. create the file (3CH) */
    regs.h.ah = 0x3C;
    regs.x.cx = 0;                                  /* normal attribute */
    regs.x.dx = (unsigned int)filename;             /* DS:DX -> filename */
    int86(0x21, &regs, &regs);
    if (regs.x.cflag) { printf("Error creating file. Code = %u\n", regs.x.ax); return; }
    handle = regs.x.ax;
    printf("File created. Handle = %u\n", handle);

    /* 2. write data (40H) */
    regs.h.ah = 0x40;
    regs.x.bx = handle;
    regs.x.cx = strlen(data);                       /* number of bytes */
    regs.x.dx = (unsigned int)data;
    int86(0x21, &regs, &regs);
    if (regs.x.cflag) { printf("Error writing file.\n"); return; }
    printf("Bytes written: %u\n", regs.x.ax);

    /* 3. close (3EH) */
    regs.h.ah = 0x3E;
    regs.x.bx = handle;
    int86(0x21, &regs, &regs);
    printf("File closed.\n");

    /* 4. open for reading (3DH, AL = 0) */
    regs.h.ah = 0x3D;
    regs.h.al = 0;
    regs.x.dx = (unsigned int)filename;
    int86(0x21, &regs, &regs);
    if (regs.x.cflag) { printf("Error opening file. Code = %u\n", regs.x.ax); return; }
    handle = regs.x.ax;
    printf("File opened for reading. Handle = %u\n", handle);

    /* 5. read (3FH) */
    regs.h.ah = 0x3F;
    regs.x.bx = handle;
    regs.x.cx = sizeof(buffer) - 1;                 /* leave room for the '\0' */
    regs.x.dx = (unsigned int)buffer;
    int86(0x21, &regs, &regs);
    if (regs.x.cflag) { printf("Error reading file.\n"); return; }
    buffer[regs.x.ax] = '\0';                       /* AX = bytes actually read */
    printf("Data read: %s\n", buffer);

    /* 6. close again */
    regs.h.ah = 0x3E;
    regs.x.bx = handle;
    int86(0x21, &regs, &regs);
}
```

Expected output (handle numbers can differ, usually 5 on a fresh DOSBox)

```
File created. Handle = 5
Bytes written: 28
File closed.
File opened for reading. Handle = 5
Data read: Hello DOS file operations!
```

Points to remember: (1) The filename and buffers are passed as near addresses, which works in Turbo C's small memory model. (2) dos.h and int86 exist only in DOS compilers such as Turbo C, not in gcc. (3) The manual's check 'handle == 0xFFFF' is not how DOS reports errors; use the carry flag. (4) Writing with DOS directly (3CH/40H) is what fopen()/fprintf() do internally, this experiment shows that layer.

One-page cheat sheet (read this last)

Topic	Remember
DIV BL	AX / BL: quotient in AL, remainder in AH. Clear AH first.
MUL BL	AX = AL x BL (unsigned).
ASCII digits	'0' = 30H. Number to character: add 30H. Character to number: subtract 30H.
INT 21H	01H read char (AL), 02H print char (DL), 09H print string (DX, ends with $), 4CH exit.
String regs	DS:SI source, ES:DI destination, CX counter, CLD = increment, STD = decrement.
Booth	10 = A-M, 01 = A+M, 00/11 = shift only. Then ASR on A,Q,Q-1. Product = A Q.
Restoring division	Shift left, A = A - M, negative: Q0 = 0 and restore, else Q0 = 1. Q = quotient, A = remainder.
Direct mapping	Line = block MOD lines. Tag = block / lines. Offset = address MOD block size.
INT 10H graphics	AX=0013H mode 13H (320x200), AH=0CH pixel with AL = colour, CX = x, DX = y, BH = page. AX=0003H = text mode.
File INT 21H	3CH create, 3DH open, 3FH read, 40H write, 3EH close. CF = 1 means error, AX = error code.